

- i) Write a C program to simulate the following CPU scheduling algorithm to find turnaround time and waiting time using FCFS.

```
#include <stdio.h>
```

```
int main() {
```

```
    int n, i;
```

```
    printf ("Enter number of processes: ");
```

```
    scanf ("%d", &n);
```

```
    int at[n], bt[n], wt[n], tat[n], rt[n],  
        et[n];
```

```
    for (i = 0; i < n; i++)
```

```
    {
```

```
        printf ("\nProcess %d\n", i+1);
```

```
        printf ("Arrival Time: ");
```

```
        scanf ("%d", &at[i]);
```

```
        printf ("Burst Time: ");
```

```
        scanf ("%d", &bt[i]);
```

```
    }
```

```
    ct[0] = at[0] + bt[0];
```

```
    tat[0] = ct[0] - at[0];
```

```
    wt[0] = tat[0] - bt[0];
```

```
    rt[0] = wt[0];
```

```
    for (i = 1; i < n; i++)
```

```
    { if (ct[i-1] < at[i])
```

```
        ct[i] = at[i] + bt[i];
```

```
    else
```

```
        ct[i] = ct[i-1] + bt[i];
```

$tot[i] = ct[i] - at[i];$
 $wt[i] = tot[i] - bt[i];$
 $rt[i] = wt[i];$
 }

printf("\n Process\t\t AT\t\t BT\t\t CT\t\t WT\t\t TAT\t\t RT\n");

float total_wt = 0, total_tat = 0, total_rt = 0;

for (i = 0; i < n; i++)

{

printf("P%d\t\t %d\t\t %d\t\t %d\t\t %d\t\t %d\t\t %d\n",
 i+1, at[i], bt[i], ct[i], wt[i], tot[i],
 rt[i]);

total_wt += wt[i];

total_tat += tot[i];

total_rt += rt[i];

}

printf("\n Average Waiting Time = %.2f", total_wt/n);
 printf("\n Average Turnaround Time = %.2f", total_tat/n);
 printf("\n Average Response Time = %.2f", total_rt/n);

return 0;

}

Enter the output
Process 1 number of processes : 4

Arrival Time : 0

Burst Time : 20

Process 2

Arrival Time : 0

Burst Time : 4

Process 3

Arrival Time : 0

Burst Time : 6

Process 4

Arrival Time : 0

Burst Time : 8

Process	AT	BT	CT	WT	TAT	RT
P1	0	2	2	0	2	0
P2	0	4	6	2	6	2
P3	0	6	12	6	12	6
P4	0	8	20	12	20	12

Average Waiting Time = 5.00

Average Turnaround Time = 10.00

Average Response Time = 5.00

output

Enter number of processes : 4

Process 1

Arrival Time : 0

Burst Time : 2

Process 2

Arrival Time : 0

Burst Time : 4

Process 3

Arrival Time : 0

Burst Time : 6

Process 4

Arrival Time : 0

Burst Time : 8

Process	AT	BT	CT	WT	TAT	RT
P1	0	2	2	0	2	0
P2	0	4	6	2	6	2
P3	0	6	12	6	12	6
P4	0	8	20	12	20	12

Average WT = 5.00

Average TAT = 10.00

Average RT = 5.00


```

printf("\nAverage WT = %0.2f", total-wt/n);
printf("\nAverage TAT = %0.2f", total-tat);
printf("\nAverage RT = %0.2f", total-rt);
return 0;
}

```

output

Enter number of Processes :- 4

Process 1

Arrival time : 0

Burst Time : 2

Process 2

Arrival time : 0

Burst Time : 4

Process 3

Arrival time : 0

Burst time : 6

Process 4

Arrival time : 0

Burst time : 8

Process	AT	BT	CT	WT	TAT	RT
P1	0	2	2	0	2	0
P2	0	4	6	2	6	2
P3	0	6	12	6	12	6
P4	0	8	20	12	20	12

Average WT = 5.00

Average TAT = 10.00

Average RT = 5.00

Comparing FCFS and SJF

- 1) FCFS executes processes in arrival order, without considering burst time
- 2) SJF selects the process with the shortest burst time, reducing waiting time
- 3) The average waiting time & turn around time in SJF are usually smaller than FCFS
- 4) Preemptive SJF gives better performance than non-preemptive SJF because it always runs the process with the shortest remaining time
- 5) FCFS is simple but less efficient, while SJF is more efficient but complex to implement.


```
printf("\n --- Preemptive Priority Scheduling ---\n");
preemptive Priority (p, n);
print Result (p, n);

return 0;
}
```

output

Enter no. of processes: 3

Enter AT, BT, Pr for P₁: 0 5 2

Enter AT, BT, Pr for P₂: 2 6 1

Enter AT, BT, Pr for P₃: 3 8 3

--- None Preemptive Priority Scheduling ---

Gantt Chart:

1) | P₁ | P₂ | P₃ |

PID	AT	BT	PR	CT	TAT	WT	RT
1	0	5	2	5	5	0	0
2	2	6	1	11	9	3	3
3	3	8	3	19	16	8	8

2) | P₁ | P₂ | P₂ | P₁ | P₃ |
0 5-2=3 2 3 8 3 11 19

Average TAT = 10.00

Average WT = 3.67

Average RT = 3.67

--- preemptive Priority Scheduling ---

Cantt chart:

~~P1~~ P1 P2 P2 P2 P2 P2 P2 P2 P1 P1 P1
~~P3~~ P3 P3 P3 P3 P3 P3 P3 P3 P3 P3
 1 P1 | P2 | P1 | P3 |

PID	AT	BT	PR	CT	TAT	WT	RT
1	0	5	2	11	U	6	0
2	2	6	1	8	6	0	0
3	3	8	3	19	16	8	8

Average TAT = 11.00

Average WT = 4.67

Average RT = 2.67

! heavy thread

1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20

PID	AT	BT	PR	CT	TAT	WT	RT
1	0	5	2	11	U	6	0
2	2	6	1	8	6	0	0
3	3	8	3	19	16	8	8

1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20


```

float avgTAT = 0, avgWT = 0;
printf("\n\nPID\tAT\tBT\tCT\tTAT\tWT\n");

for (int i=0; i<n; i++)
{
    printf("Process %d, AT=%d, BT=%d, CT=%d, TAT=%d, WT=%d\n",
           p[i].pid, p[i].at, p[i].bt, p[i].ct, p[i].tat, p[i].wt);

    avgTAT += p[i].tat;
    avgWT += p[i].wt;
}

printf("\n Average TAT = %.2f", avgTAT/n);
printf("\n Average WT = %.2f", avgWT/n);

return 0;
}
    
```

Output

Enter number of processes :- 4

Process 1	Process 2	Process 3	Process 4
AT : 0	AT : 0	AT : 3	AT : 4
BT : 2	BT : 5	BT : 4	BT : 5

Time Quantum :- 3

Grant chart

PID	AT	BT	CT	TAT	WT
P1	0	2	2	2	0
P2	0	5	13	13	8
P3	3	4	14	11	7
P4	4	5	16	12	7

Average TAT = 9.50

Average WT = 5.50

Time Quantum = 4

Gantt chart

| P1 | P2 | P3 | P1 | P2 | P1 |

PID	AT	BT	CT	TAT	WT
P1	0	2	2	2	0
P2	0	5	15	15	10
P3	3	4	10	7	3
P1	4	5	16	12	7

Average TAT = 9.00

Average WT = 5.00

See
RD


```
printf("\n Avg TAT = %.2f", avgTAT/n);  
printf("\n Avg WT = %.2f", avgWT/n);  
printf("\n Avg RT = %.2f", avgRT/n);  
return 0;
```

Enter the no. of Processes : 4

Enter Arrival time, Burst time, Type(system, User):
5 5 system

Enter Arrival time, Burst time, type(system, user):
0 5 user.

Enter Arrival time, Burst time, type(system, user):
1 5 user

Enter Arrival time, Burst time, type(system, user):
2 5 user.

P2	P1	P3	P4
0	5	10	14

Process	type	AT	BT	CT	TAT	WT	RT
P2	User	0	5	5	5	0	0
P3	User	1	5	15	14	9	9
P4	User	2	5	20	18	13	13
P5	System	5	5	10	5	0	0

$$\text{Avg TAT} = 10.50$$

$$\text{Avg WT} = 5.50$$

$$\text{Avg RT} = 5.50$$


```

printf("\n Process no. : \n");
printf("Burst time:");
scanf("%d", &p[i].bt);
printf("Deadline (for EDF):");
scanf("%d", &p[i].deadline);
printf("Period (for RMS): \n");
scanf("%d", &p[i].period);
}

```

```

check_schedulability(p, n);
for (int i = 0; i < n; i++) temp[i] = p[i];
edf(temp, n);
for (int i = 0; i < n; i++) temp[i] = p[i];
rms(temp, n);
return 0;
}

```

Output

Enter no. of processes : 3

Enter process details.

process 0:

Burst Time : 1

Deadline (for EDF) : 5

Period (for RMS) : 5

Process 1:

BT : 2

Deadline : 3

Period : 10

Process 2:

BT : 1

Deadline : 5

Period : 10

Schedulability Test
Total Utilization (U) = 0.500

(RMS)

Bound = 0.780

Schedulable under RMS

(EDF)

Bound = 0.780

Schedulable under EDF

= = EDF - - - =

Grantt chart

| P2 | P1 | P3 |
0 2 3 4

ID	Deadline	CT	WT	TAT
1	5	3	2	3
2	3	2	0	2
3	5	4	3	4

= = RMS - - - =

Grantt chart

| P1 | P2 | P3 |
0 1 3 4

ID	BT	Period	CT	WT	TAT
1	1	5	1	0	1
2	2	10	3	1	3
3	1	10	4	3	4


```
printf("\n Average Turnaround Time = %d.2f total wt / n\n");
printf("\n Average wt = %d.2f total wt (n);
```

```
return 0;
```

```
}
```

output

Enter no. of processes 3

Process 1

BT = 1

weight = 5

Process 2

BT = 1

weight = 5

Process 3

BT = 2

weight = 3

Gantt chart

| P1 | P2 | P3 |

Process	BT	weight	CT	TAT	wt
P1	1	5	1	1	0
P2	1	5	2	2	1
P3	2	3	4	4	2

Avg TAT = 2.33 ("n/d")

Avg wt = 1.00

Practice - 2

The following program consists of 3 Concurrent process and 3 binary Semaphores. The semaphores are initialize as $S_0 = 1$ $S_1 = 0$ $S_2 = 0$. find out how many P_0 will print 0. Assume order of execution as P_0, P_1, P_2, P_0, P_1 .

Process P_0

```
while (true)
{ wait( $S_0$ )
  print (0)
  signal( $S_2$ )
  signal( $S_1$ )
}
```

Process P_1 ;

```
while (true)
{ wait( $S_1$ )
  signal( $S_0$ );
}
```

Process P_2 ;

```
{ wait( $S_2$ );
  signal( $S_0$ );
}
```

initial	$S_0 = 1$	$S_1 = 0$	$S_2 = 0$	remarks
P_0	0	1	1	0
P_1	1	0	1	-
P_2	1	0	0	-
P_0	0	1	1	0 0
P_1	1	0	1	-

P_0 prints 0 2 times.


```

pthread_join(c, NULL);
sem_destroy(&empty);
sem_destroy(&full);
pthread_mutex_destroy(&mutex);
free(buffer);
return 0;
}
  
```

Output

Enter buffer size = 4

Enter no. of items to produce :- 3

produced : 1 | buffer [1 0 0 0]

Consumed : 1 | buffer [0 0 0 0]

produced : 2 | buffer [0 2 0 0]

Consumed : 2 | Buffer [0 0 0 0]

produced : 3 | buffer [0 0 3 0]

Consumed : 3 | buffer [0 0 0 0]

Output

philosopher 2 is Thinking
philosopher 4 is Thinking
philosopher 3 is Thinking
philosopher 1 is Thinking
philosopher 1 picked Left fork
philosopher 4 picked Right fork
philosopher 4 picked Left fork
philosopher 4 is Eating
philosopher 2 picked Right fork
philosopher 1 picked Right fork
philosopher 1 is Eating
philosopher 1 finished eating & exits
philosopher 0 picked Right fork
philosopher 0 picked Left fork
philosopher 0 is Eating
philosopher 2 picked Left fork
philosopher 2 is Eating
philosopher 4 finished eating & exits
philosopher 0 finished eating & exits
philosopher 3 picked Left fork
philosopher 3 picked Right fork
philosopher 3 is Eating
philosopher 2 finished eating & exits
philosopher 3 finished eating & exits.

All philosophers have eaten once, program ends

see


```
#include <stdio.h>
```

```
#define Max 10
```

```
int main()
```

```
{ int n, m;
```

```
printf("Enter number of process:");
```

```
scanf("%d", &n);
```

```
printf("Enter number of resources:");
```

```
scanf("%d", &m);
```

```
int alloc[Max][Max], max[Max][Max], need[Max][Max];
```

```
int avail[Max], finish[Max] = {0};
```

```
int safeSeq[Max];
```

```
printf("\nEnter Allocation Matrix: \n");
```

```
for (int i = 0; i < n; i++)
```

```
{ for (int j = 0; j < m; j++)
```

```
{ scanf("%d", &alloc[i][j]);
```

```
}
```

```
}
```

```
printf("\nEnter Maximum Matrix: \n");
```

```
for (int i = 0; i < n; i++)
```

```
{ for (int j = 0; j < m; j++)
```

```
{ scanf("%d", &max[i][j]);
```

```
}
```

```
}
```

```
printf("\nEnter Available Resources: \n");
```

```
for (int i = 0; i < m; i++)
```

```
{ scanf("%d", &avail[i]);
```

```
printf("\nEnter Need Matrix: \n");
```

```
for (int i = 0; i < n; i++)
```

```
{ for (int j = 0; j < m; j++)
```

```
{ need[i][j] = max[i][j] - alloc[i][j];
```

Output

(i) Enter number of processes:- 4

Enter number of resources:- 3

Enter Allocation Matrix Enter Maximum Matrix

0 1 0 7 5 3

2 0 0 3 2 2

3 0 2 9 0 2

2 1 1 2 2 2

Enter Available Resources:-

~~2 3 2~~

Enter Need Matrix

7 4 3

1 2 2

6 0 0

0 1 1

System is in safe state

Safe sequence :- P1 | P3 | ~~P2~~ | P2

(ii) Allocation Max Allocation

0 1 0 7 5 3 0 0 0

2 0 0 3 2 2

3 0 2 9 0 2

2 1 1 2 2 2

System is not in safe state.

Output

Enter number of processes: 4

Enter number of Resource: 3

Enter Available resources: 3 3 2

Enter Allocation

Enter Maximum

Need matrix

0 1 0

7 5 3

7 6 3

2 0 0

3 2 2

1 2 2

3 0 2

9 0 2

6 0 0

2 1 1

2 2 2

0 1 1

System is in Safe State

Safe sequence: P1 | P3 | P0 | P2

Enter process number making request: 1

Enter request vector

1 0 2

Request can be granted, System remains Safe
Safe sequence: P0, P1, P2, P3

Enter process number making request: 1

Enter request vector

4 4 3

Request exceeds need (Error)

Output

- (i) Enter no. of processes : 5
Enter no. of resource : 3

Allocation	Request	Available
0 1 0	0 0 0	0 0 0
2 0 0	0 0 2	
3 0 3	0 0 0	
2 1 1	1 0 0	
0 0 2	0 0 2	

System is in safe state

Safe sequence P0 P2 P3 P4 P1

(ii)

Allocation	Request	Available
0 1 0	1 1 1	0 0 0
2 0 0	2 0 2	
3 0 3	1 0 1	
2 1 1	1 0 0	
0 0 2	0 0 2	

System is ~~not~~ not in a safe state.

Output

Enter size of each memory block :- 400 700 200 300 600

Enter no. of processes 4

Enter size of each process :- 212 512 312 526

First Fit

Process No.	Process Size	Block No.
1	212 KB	1
2	512 KB	2
3	312 KB	5
4	526 KB	Not Allocated.

Best Fit

Process No.	Process Size	Block No.
1	212 KB	4
2	512 KB	5
3	312 KB	1
4	526 KB	2

Worst Fit

Process No.	Process Size	Block No.
1	212 KB	2
2	512 KB	5
3	312 KB	1
4	526 KB	not Allocated.

(Signature)